

Reviewer Pack — Matroid Rank Submodularity in Monotone DNF for k-CLIQUE

Entry 16b4e4e9c316 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 07:54:20 UTC

Matroid Rank Submodularity in Monotone DNF for k-CLIQUE

Entry ID: 16b4e4e9c316 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 07:54:20 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory (Polymatroid Expansion) **Field B** (complexity object): Monotone Circuit Size for k-CLIQUE

Statement:

Let $\mu(F)$ be the rank of the transversal matroid induced by a monotone DNF formula F . For all F with size $\text{poly}(n)$, $\mu(F) = O(\log n)$. For k-CLIQUE instances, $\mu(F) = \Omega(n^{1 - 1/k})$

Rationale (proposer’s reasoning):

Transversal matroid rank captures submodularity of DNF terms, aligning with monotone circuit structure. k-CLIQUE’s combinatorial constraints force high rank due to clique edge intersections, creating a gap between general DNFs and clique-specific formulas.

Taxonomy category: MONOTONE_CLIQUE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cdf5370e30f964bf

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def generate_dnf(n: int) -> list:
    dnf = set()
    for i in range(1, n + 1):
        terms = [set() for _ in range(i)]
        for j in range(n):
            if random.choice([True, False]):
                terms[j % i].add(j)
        dnf.add(tuple(sorted(term) for term in terms))
    return list(dnf)

def max_matching_size(dnf: list) -> int:
    n = len(dnf[0])
    adj_matrix = [[0] * n for _ in range(n)]
    for clause in dnf:
        for i in range(n):
            if i not in clause:
                continue
            for j in range(i + 1, n):
                if j not in clause:
                    adj_matrix[i][j] += 1
                    adj_matrix[j][i] += 1

    matching = []
    visited = [False] * n

    def dfs(u: int) -> bool:
        for v in range(n):
            if adj_matrix[u][v] > 0 and not visited[v]:
                visited[v] = True
                if v not in matching or dfs(matching[v]):
                    matching[v] = u
                    return True
        return False

    for i in range(n):
        if not visited[i]:
            dfs(i)

    return sum(1 for x in matching if x != -1)

def run_trial(seed: int) -> dict:
```

```

random.seed(seed)
n = 40
dnf_size = n**2
k = 3 # Example value for k-CLIQUE

dnf = generate_dnf(n)
mu_F = max_matching_size(dnf)

if len(dnf[0]) == n:
    mu_F_k_clique = sum(1 for _ in combinations(range(n), k))
else:
    mu_F_k_clique = -1

metric_name = "mu_F"
metric_value = mu_F
instances_tested = 1
conjecture_holds = False
counterexample = ""

if len(dnf[0]) == n:
    if mu_F >= n*(1 - 1/k):
        conjecture_holds = True
    else:
        counterexample = "k-CLIQUE instance does not meet the conjecture"
elif mu_F <= math.log(n):
    conjecture_holds = True
else:
    counterexample = "General DNF does not meet the conjecture"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    else:
        print(f"RESULT: NOT SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")

```

```

first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}\")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c7104728.py", line 101, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c7104728.py", line 63, in run_trial
    dnf = generate_dnf(n)
          ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c7104728.py", line 24, in generate_dnf
    dnf.add(tuple(sorted(term) for term in terms))
TypeError: unhashable type: 'list'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to unhashable type error, preventing data collection to evaluate conjecture | next: Fix generate_dnf() to handle hashable types and re-run experiments

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	47478
2	preregistration	ollama_remote	qwen3:8b	0	0	24283
3	novelty	ollama_remote	qwen3:8b	0	0	20812
4	novelty	ollama_remote	qwen3:8b	0	0	13702
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13319
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11000
7	judge	ollama_remote	qwen3:8b	0	0	17783

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 148376 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/16b4e4e9c316.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/16b4e4e9c316.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/16b4e4e9c316.tar.gz` (if generated)